Breadth-First Search (BFS)

Lecture : Graph Traversal, Queue Operations, Dry Runs, Shortest Paths and C Implementation

Department of Computer Science and Engineering

Graphic Era (Deemed to be University)

Recap: Graph Representation

Today's Question

How can we systematically visit vertices in a graph?

What Is Graph Traversal?

Definition

Graph traversal is the process of systematically visiting vertices of a graph according to a defined strategy.

Two fundamental traversal techniques are:

1. Breadth-First Search (BFS).
2. Depth-First Search (DFS).

Application

Explore all devices reachable from a computer in a communication network.

Why Do We Need Graph Traversal?

Graph traversal helps solve problems such as:

- Finding reachable vertices.
- Searching for a particular vertex.
- Discovering connected components.
- Finding shortest paths.
- Exploring computer networks.
- Finding relationships in social networks.
- Building spanning trees.

Breadth-First Search: Definition

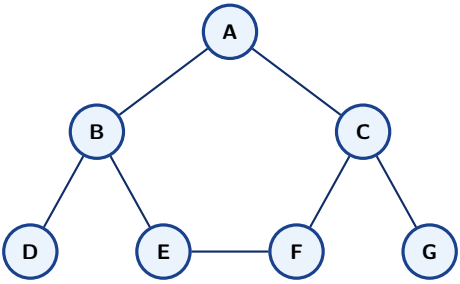
Definition

Breadth-First Search is a graph traversal algorithm that explores vertices in increasing order of their distance, measured in edges, from a starting vertex.

- Starts from a selected source.
- Visits immediate neighbors first.
- Then visits neighbors of those neighbors.
- Uses a queue.
- Marks discovered vertices to avoid repetition.

Basic Idea of BFS

BFS explores a graph level by level.



Level 0: A

Level 1: B, C

Level 2: D, E, F, G

BFS vs Depth-Oriented Exploration

BFS

Explores vertices level by level.
Uses a queue.

DFS

Explores one branch deeply before backtracking.
Uses recursion or a stack.

Important

BFS is particularly useful for finding minimum-edge paths in unweighted graphs.

Why Does BFS Use a Queue?

A queue follows:

FIFO

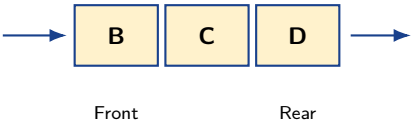
First In, First Out.

- Newly discovered vertices are enqueued.
- The earliest discovered pending vertex is processed first.
- This maintains level-by-level exploration.

Queue Operations

Enqueue: Insert at rear.
Dequeue: Remove from front.

Queue: Visual Representation



Processing Order

$$B \rightarrow C \rightarrow D$$

B is removed before C and D.

BFS: Three Important Data Structures

1. **Graph representation**
   Adjacency matrix or adjacency list.
2. **Queue**
   Stores discovered vertices waiting to be processed.
3. **Visited array**
   Tracks whether each vertex has already been discovered.

$$visited[v] \in \{0, 1\}$$

Why Is a Visited Array Necessary?

Consider a cycle:

$$A \to B \to C \to A$$

Without visited tracking:

$$A, B, C, A, B, C, \ldots$$

The traversal may continue indefinitely.

Solution

Mark each vertex as visited when it is first enqueued.

This prevents duplicate queue entries and repeated exploration.

When Should a Vertex Be Marked?

Correct Strategy

Mark a vertex as visited when it is inserted into the queue.

1. Discover an unvisited neighbor.
2. Mark it as visited.
3. Enqueue it.

Common Mistake

If marking is delayed until dequeue, the same vertex may be enqueued multiple times by different neighbors.

BFS Algorithm: Main Steps

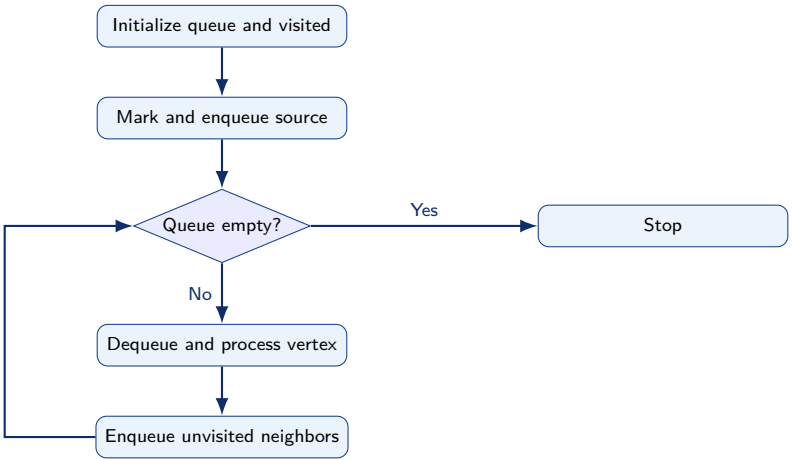
1. Select the starting vertex.
2. Mark it as visited.
3. Insert it into the queue.
4. Remove the front vertex.
5. Process the removed vertex.
6. Examine its adjacent vertices.
7. Enqueue every newly discovered neighbor.
8. Repeat until the queue is empty.

BFS Pseudocode

Algorithm: BFS(G, start)

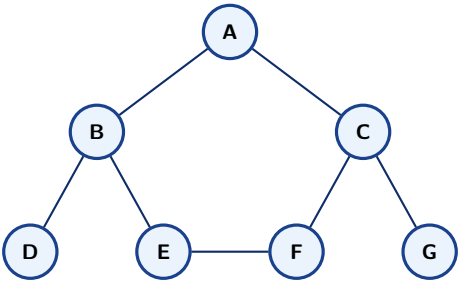
```
1.  Mark all vertices unvisited
2.  Create an empty queue
3.  visited[start] = true
4.  Enqueue(start)
5.  While queue is not empty:
  u = Dequeue()
  Process(u)
  For each neighbor v of u:
    If visited[v] is false:
      visited[v] = true
      Enqueue(v)
```

BFS Flowchart



Worked Example: BFS from A

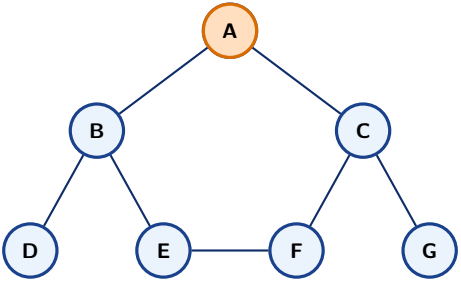
Consider the graph:



Question

Perform BFS starting from A.
Process adjacent vertices in alphabetical order.

Step 0: Initialize BFS



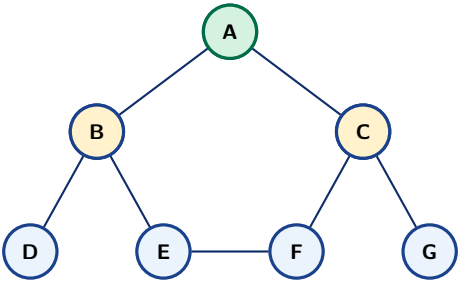
Initialization

$$visited = \{A\}$$

$$Queue = [A]$$

$$Traversal = []$$

Step 1: Process A

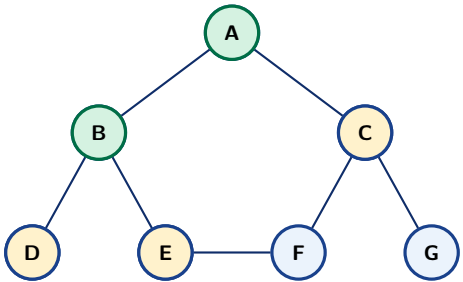


1. Dequeue A.
2. Visit A.
3. Discover B and C.
4. Enqueue B and C.

$$Queue = [B, C]$$

$$Traversal = [A]$$

Step 2: Process B

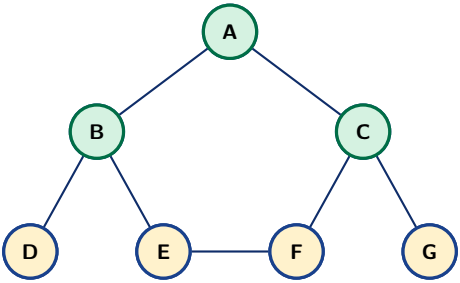


1. Dequeue B.
2. A is already visited.
3. Discover D and E.
4. Enqueue D and E.

$$Queue = [C, D, E]$$

$$Traversal = [A, B]$$

Step 3: Process C

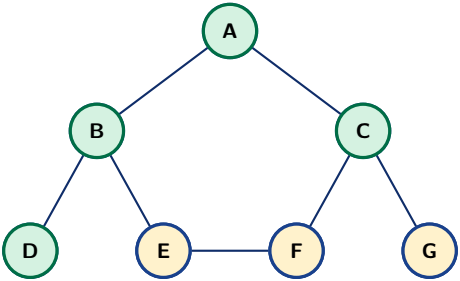


1. Dequeue C.
2. A is already visited.
3. Discover F and G.
4. Enqueue F and G.

$$Queue = [D, E, F, G]$$

$$Traversal = [A, B, C]$$

Step 4: Process D

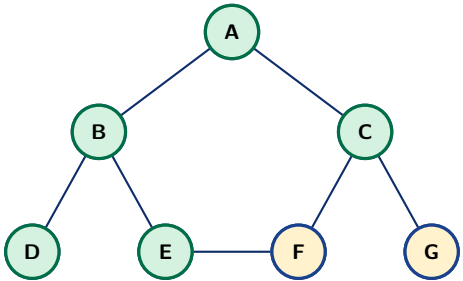


D has only one neighbor: B.
B is already visited.

$$Queue = [E, F, G]$$

$$Traversal = [A, B, C, D]$$

Step 5: Process E

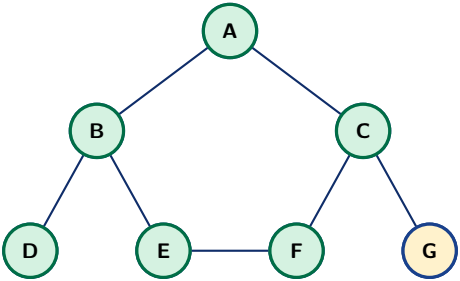


E is adjacent to B and F.
Both have already been discovered.

$$Queue = [F, G]$$

$$Traversal = [A, B, C, D, E]$$

Step 6: Process F

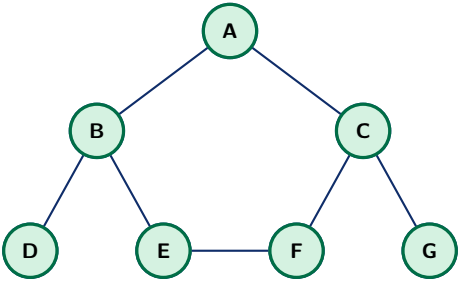


F is adjacent to C and E.
Both are already visited.

$$Queue = [G]$$

$$Traversal = [A, B, C, D, E, F]$$

Step 7: Process G



G is adjacent to C.
C is already visited.

$$Queue = []$$

$$Traversal = [A, B, C, D, E, F, G]$$

Complete BFS Dry Run

Step	Dequeued	Queue After Processing	Traversal
0	–	A	–
1	A	B,C	A
2	B	C,D,E	A,B
3	C	D,E,F,G	A,B,C
4	D	E,F,G	A,B,C,D
5	E	F,G	A,B,C,D,E
6	F	G	A,B,C,D,E,F
7	G	Empty	A,B,C,D,E,F,G

Final BFS Order

$$\boxed{A, B, C, D, E, F, G}$$

BFS Traversal Order Depends on Neighbors

BFS traversal order may change when adjacency-list ordering changes.
For example, starting at A:

$$A, B, C, D, E, F, G$$

or:

$$A, C, B, F, G, D, E$$

may both be valid for the same graph under different neighbor-order conventions.

Important

BFS preserves increasing distance from the source, but the order within a level can vary.

What Is a BFS Tree?

Definition

A BFS tree is formed by the edges through which vertices are first discovered.

For our example, the discovery edges are:

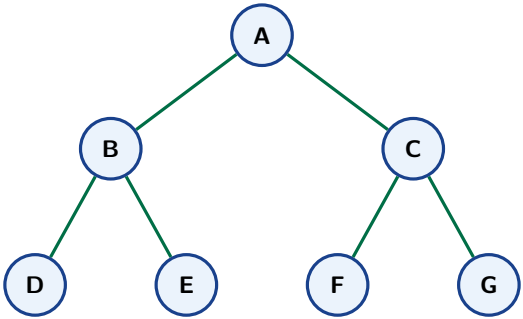
$$(A, B),\ (A, C)$$

$$(B, D),\ (B, E)$$

$$(C, F),\ (C, G)$$

The edge (E, F) is not a BFS tree edge.

BFS Tree: Diagram



Observation

The BFS tree contains all seven vertices and six discovery edges.

BFS Levels

Level	Vertices	Distance from A
0	A	0
1	B,C	1
2	D,E,F,G	2

Property

BFS discovers vertices in nondecreasing order of their shortest-path distance from the source.

BFS Distance Array

For the example graph:

$$dist[A] = 0$$

$$dist[B] = dist[C] = 1$$

$$dist[D] = dist[E] = dist[F] = dist[G] = 2$$

Interpretation

The shortest path from A to G uses two edges:

$$A \to C \to G$$

BFS with an Adjacency Matrix

For a graph with V vertices:

$$A[V][V]$$

To find neighbors of vertex u, scan row u:

$$A[u][0], A[u][1], \ldots, A[u][V-1]$$

Important

If:

$$A[u][v] = 1$$

then v is adjacent to u.

Adjacency Matrix Example

Consider vertices:

$$A, B, C, D$$

and edges:

$$(A, B), (A, C), (B, D), (C, D)$$

The matrix is:

$$\begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{bmatrix}$$

Starting at A, BFS visits:

$$\boxed{A, B, C, D}$$

C Program: Adjacency Matrix BFS

This implementation uses:

- A fixed-size adjacency matrix.
- A queue implemented using an array.
- A visited array.
- A starting vertex supplied to BFS.

Vertex Mapping

$$0 = A, \quad 1 = B, \quad 2 = C, \quad 3 = D$$

C Program: Matrix BFS, Part 1

```c
#include <stdio.h>

#define MAX 10

int graph[MAX][MAX];
int visited[MAX];
int queue[MAX];
int front, rear;
int vertices;

void addEdge(int u, int v) {
    graph[u][v] = 1;
    graph[v][u] = 1;
}

void bfs(int start) {
    front = 0;
    rear = 0;

    for (int i = 0; i < vertices; i++)
        visited[i] = 0;

    visited[start] = 1;
    queue[rear++] = start;

    printf("BFS Traversal: ");
```

C Program: Matrix BFS, Part 2

```
    while (front < rear) {
        int u = queue[front++];

        printf("%c ", 'A' + u);

        for (int v = 0; v < vertices; v++) {
            if (graph[u][v] == 1 &&
                visited[v] == 0) {

                visited[v] = 1;
                queue[rear++] = v;
            }
        }
    }

    printf("\n");
}
```

Observation

The queue never stores more than V vertices because each vertex is enqueued at most once.

C Program: Matrix BFS, Main

```c
int main(void) {
    vertices = 4;

    addEdge(0, 1);
    addEdge(0, 2);
    addEdge(1, 3);
    addEdge(2, 3);

    bfs(0);

    return 0;
}
```

Expected Output

```
BFS Traversal:  A B C D
```

Matrix BFS: Code Explanation

1. Start vertex A is marked visited.
2. A is enqueued.
3. A is dequeued and printed.
4. Matrix row A is scanned.
5. B and C are discovered and enqueued.
6. Their rows are scanned in FIFO order.
7. D is discovered.
8. Traversal ends when the queue is empty.

Matrix BFS: Complexity

For V vertices:

- Each vertex is processed once.
- Processing a vertex scans an entire matrix row.
- Each row contains V entries.

Therefore:

$$\boxed{T(V) = O(V^2)}$$

Auxiliary queue and visited arrays:

$$\boxed{O(V)}$$

Matrix storage:

$$\boxed{O(V^2)}$$

BFS with an Adjacency List

An adjacency list stores only the neighbors of each vertex.

Example

```
A: B -> C
B: A -> D
C: A -> D
D: B -> C
```

BFS examines only the entries present in each adjacency list.

Why Adjacency Lists Are Efficient

For a graph with:

V vertices, E edges

BFS:

- Processes each vertex at most once.
- Examines each adjacency entry.
- Uses a queue for pending vertices.

$$\boxed{T(V, E) = O(V + E)}$$

for a full traversal of the graph.

C Program: Adjacency List BFS

We will implement BFS using:

- Linked-list adjacency representation.
- Dynamic memory allocation.
- Array-based queue.
- Visited array.

Example Graph

$$E = \{(A, B), (A, C), (B, D), (C, D)\}$$

C Program: List BFS, Part 1

```c
#include <stdio.h>
#include <stdlib.h>

#define MAX 10

typedef struct Node {
    int vertex;
    struct Node *next;
} Node;

Node *adj[MAX];
int visited[MAX];
int queue[MAX];
int vertices;

Node *createNode(int v) {
    Node *p = malloc(sizeof(Node));

    if (p == NULL) {
        perror("Memory allocation failed");
        exit(EXIT_FAILURE);
    }

    p->vertex = v;
    p->next = NULL;

    return p;
}
```

C Program: List BFS, Part 2

```c
void initialize(int n) {
    vertices = n;

    for (int i = 0; i < n; i++) {
        adj[i] = NULL;
        visited[i] = 0;
    }
}

void addEdge(int u, int v) {
    Node *p = createNode(v);
    p->next = adj[u];
    adj[u] = p;

    Node *q = createNode(u);
    q->next = adj[v];
    adj[v] = q;
}
```

Note

Neighbors are inserted at the head of each adjacency list.

C Program: List BFS, Part 3

```c
void bfs(int start) {
    int front = 0;
    int rear = 0;

    for (int i = 0; i < vertices; i++)
        visited[i] = 0;

    visited[start] = 1;
    queue[rear++] = start;

    printf("BFS Traversal: ");

    while (front < rear) {
        int u = queue[front++];

        printf("%c ", 'A' + u);

        Node *p = adj[u];

        while (p != NULL) {
            int v = p->vertex;

            if (!visited[v]) {
                visited[v] = 1;
                queue[rear++] = v;
            }

            p = p->next;
        }
    }

    printf("\n");
}
```

C Program: List BFS, Part 4

```c
void freeGraph(void) {
    for (int i = 0; i < vertices; i++) {
        Node *p = adj[i];

        while (p != NULL) {
            Node *temp = p;
            p = p->next;
            free(temp);
        }

        adj[i] = NULL;
    }
}
```

Memory Management

All dynamically allocated adjacency-list nodes are released.

C Program: List BFS, Main

```c
int main(void) {
    initialize(4);

    addEdge(0, 1);
    addEdge(0, 2);
    addEdge(1, 3);
    addEdge(2, 3);

    bfs(0);

    freeGraph();

    return 0;
}
```

Expected Output

```
BFS Traversal:  A C B D
```

Why This Order?

Head insertion causes C to appear before B in A's list.

Adjacency Matrix vs List BFS

Feature	Matrix BFS	List BFS
Time	$O(V^2)$	$O(V + E)$
Graph Storage	$O(V^2)$	$O(V + E)$
Neighbor Access	Scan entire row	Scan neighbors only
Sparse Graph	Less efficient	Usually preferable
Dense Graph	Often suitable	Also possible

Problem: Disconnected Graph

Consider:



Question

If BFS starts at A, will it visit D and E?

Answer

No. A single-source BFS visits only vertices reachable from its source.

Solution: BFS Forest

To traverse every vertex:

1. Mark all vertices unvisited.
2. Iterate over all vertices.
3. If a vertex is unvisited, start BFS from that vertex.
4. Continue until all vertices have been visited.

Result

The process creates a BFS forest containing one BFS tree per connected component in an undirected graph.

Disconnected BFS Pseudocode

Algorithm: BFSAll

```
1.  Mark all vertices unvisited
2.  For each vertex s:
  If visited[s] is false:
    Enqueue(s)
    Mark s visited
    While queue is not empty:
      u = Dequeue()
      Process(u)
      Discover and enqueue
      all unvisited neighbors
```

Disconnected Graph: Worked Example

Suppose:

$$V = \{A, B, C, D, E, F\}$$

and:

$$E = \{(A, B), (B, C), (D, E)\}$$

Connected components:

$$\{A, B, C\}, \quad \{D, E\}, \quad \{F\}$$

Full BFS Traversal

Assuming alphabetical vertex selection:

$$\boxed{A, B, C, D, E, F}$$

Three BFS trees are created.

BFS and Shortest Paths

Important Property

In an unweighted graph, BFS finds a path with the minimum number of edges from the source to every reachable vertex.

For example:

$$A \to B \to D$$

has two edges.
If no one-edge path exists, its shortest-path distance is two.

Why Does BFS Find Shortest Paths?

BFS uses a FIFO queue.

1. The source is discovered at distance 0.
2. Its neighbors are discovered at distance 1.
3. Their undiscovered neighbors are found at distance 2.
4. This continues level by level.

Key Property

The first discovery of a vertex gives its minimum number of edges from the source.

Distance and Parent Arrays

To reconstruct shortest paths, maintain:

$$dist[v]$$

and:

$$parent[v]$$

- `dist[v]` stores shortest-path distance.
- `parent[v]` stores the predecessor through which v was first discovered.

When discovering neighbor v from u:

$$dist[v] = dist[u] + 1$$

$$parent[v] = u$$

Shortest Path Example

Consider:

$$A - B, \quad A - C, \quad B - D, \quad C - E, \quad D - F$$

Starting from A:

Vertex	Distance	Parent
A	0	–
B	1	A
C	1	A
D	2	B
E	2	C
F	3	D

$$A \to B \to D \to F$$

is a shortest path to F.

Path Reconstruction

To reconstruct a path to target t:

1. Start at t.
2. Follow its parent pointer.
3. Continue until the source is reached.
4. Reverse the collected sequence.

Example

$$parent[F] = D$$

$$parent[D] = B$$

$$parent[B] = A$$

Result:

$$\boxed{A \to B \to D \to F}$$

BFS Does Not Solve All Weighted Shortest Paths

BFS minimizes:

Number of edges

It does not generally minimize the sum of arbitrary edge weights.

Example

A direct edge with weight 100 may be more expensive than a two-edge path with total weight 5.

For nonnegative weighted graphs, Dijkstra's algorithm is commonly used.

BFS Time Complexity

Adjacency List

$$\boxed{O(V + E)}$$

Each vertex is processed once, and adjacency entries are examined.

Adjacency Matrix

$$\boxed{O(V^2)}$$

Every processed vertex requires scanning a full matrix row.

BFS Space Complexity

BFS uses:

- Visited array: $O(V)$.
- Queue: $O(V)$.
- Optional parent array: $O(V)$.
- Optional distance array: $O(V)$.

$$\boxed{\text{Auxiliary Space} = O(V)}$$

Important

This excludes the memory used to store the graph itself.

Applications of BFS

1. Shortest paths in unweighted graphs.
2. Social-network connection levels.
3. Web crawling.
4. Network broadcasting.
5. Connected-component discovery.
6. Maze solving.
7. Bipartite-graph testing.
8. Level-order traversal of trees.

BFS in Social Networks

Consider a social network:

- Level 0: You.
- Level 1: Direct friends.
- Level 2: Friends of friends.
- Level 3: Third-degree connections.

BFS Application

Discover all users within two friendship links of a selected person.

BFS in Computer Networks

In a computer network:

- Vertices represent devices.
- Edges represent communication links.
- BFS explores reachable devices.
- Levels represent minimum hop counts.

Example

Find all devices reachable within three network hops from a server.

BFS Advantages

- Systematic level-by-level exploration.
- Complete for finite reachable graphs.
- Finds shortest paths in unweighted graphs.
- Straightforward queue-based implementation.
- Supports reachability analysis.

BFS Limitations

- Requires memory for the queue.
- May discover many vertices at one level.
- Does not directly solve arbitrary weighted shortest paths.
- A single BFS does not visit unreachable components.
- Traversal order depends on adjacency ordering.

Exercise 1: BFS Traversal

Question

Perform BFS starting from A.

$$E = \{(A, B), (A, C), (B, D), (C, E), (D, F)\}$$

Process neighbors alphabetically.

Answer

$$\boxed{A, B, C, D, E, F}$$

Exercise 2: Queue Dry Run

Question

For the graph:

$$A - B, \quad A - C, \quad B - D, \quad C - E$$

What is the queue after processing A?

Answer

$$\boxed{[B, C]}$$

After processing B:

$$\boxed{[C, D]}$$

Exercise 3: Shortest Path

Question

Given:

$$A - B, \quad B - C, \quad A - D, \quad D - E, \quad E - C$$

Find the minimum number of edges from A to C.

Answer

$$A \to B \to C$$

$$\boxed{dist(C) = 2}$$

Exercise 4: Disconnected Graph

Question

A graph has vertices:

$$\{A, B, C, D, E\}$$

and edges:

$$(A, B), (B, C), (D, E)$$

How many BFS trees are required to cover all vertices?

Answer

$$\boxed{2}$$

One for $\{A, B, C\}$, one for $\{D, E\}$.

MCQ 1

Which data structure is used in BFS?

1. Stack
2. Queue
3. Heap
4. Binary search tree

Correct Answer

Option 2: Queue.

MCQ 2

BFS explores vertices:

1. In decreasing degree order
2. In arbitrary depth-first order
3. Level by level
4. Only in sorted key order

Correct Answer

Option 3: Level by level.

MCQ 3

The time complexity of BFS using adjacency lists is:

1. $O(V^2)$
2. $O(V + E)$
3. $O(E^2)$
4. $O(\log V)$

Correct Answer

Option 2: $O(V + E)$.

MCQ 4

BFS can find shortest paths in:

1. Arbitrarily weighted graphs
2. Unweighted graphs
3. Only complete graphs
4. Only directed acyclic graphs

Correct Answer

Option 2: Unweighted graphs.

MCQ 5

When should a vertex be marked visited in standard BFS?

1. When first enqueued
2. Only after all vertices are processed
3. After the queue becomes empty
4. Never

Correct Answer

Option 1: When first enqueued.

MCQ 6

A single-source BFS on a disconnected graph visits:

1. Every vertex automatically
2. Only the source vertex
3. Vertices reachable from the source
4. Only vertices of even degree

Correct Answer

Option 3.

BFS: Important Concepts

Concept	BFS Property
Traversal strategy	Level by level
Data structure	Queue
Queue discipline	FIFO
Visited marking	On discovery
Shortest path	Minimum edge count
Adjacency-list time	$O(V + E)$
Adjacency-matrix time	$O(V^2)$
Auxiliary space	$O(V)$

Lecture Summary

- BFS is a fundamental graph traversal algorithm.
- It explores vertices in increasing distance order.
- A queue controls processing order.
- Visited tracking prevents repeated discovery.
- BFS generates a traversal tree.
- BFS finds shortest paths in unweighted graphs.
- Disconnected graphs require repeated BFS starts.
- Adjacency-list BFS takes $O(V + E)$ time.
- Adjacency-matrix BFS takes $O(V^2)$ time.

Questions for Revision

1. Define BFS.
2. Why is a queue used in BFS?
3. Explain the role of the visited array.
4. Write BFS pseudocode.
5. Perform a BFS dry run.
6. Explain BFS tree construction.
7. Implement BFS using an adjacency matrix.
8. Implement BFS using an adjacency list.
9. Explain BFS on disconnected graphs.
10. Describe shortest-path discovery using BFS.

References

1. Reema Thareja, *Data Structures Using C*, Chapter 13: Graphs.
2. Thomas H. Cormen et al., *Introduction to Algorithms*, Elementary Graph Algorithms.
3. Mark Allen Weiss, *Data Structures and Algorithm Analysis in C*.

Next Lecture

Lecture : Depth-First Search (DFS)

- DFS fundamentals.
- Stack-based traversal.
- Recursive DFS.
- Iterative DFS.
- Detailed dry runs.
- DFS traversal trees.
- Connected components.
- C implementation.
- Complexity and applications.

Share Your Feedback



Thank you!